Vulnerability Assessment Report

Imagur Security Assessment 2026-10-06

Imagur (FShare / Image Sharer)
132 Morphett Rd
Glengowrie SA 5044, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 6, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 3

1 Document Control 4

1.1 Team 4

1.2 List of Changes 4

2 Executive Summary 5

2.1 Overview 5

2.2 Identified Vulnerabilities 6

3 Methodology 7

3.1 Objective 7

3.2 Scope 7

3.3 User Accounts and Permissions 8

4 Findings 9

H1: Stored Cross-Site Scripting (XSS) via image filename 9

M1: Session cookie missing HttpOnly, Secure, and SameSite flags 10

M2: PostgreSQL service exposed to the internet without TLS 11

M3: User account enumeration via registration 12

M4: Broken Access Control / Mass Assignment via user_id on upload 13

M5: Cross-Site Request Forgery (CSRF) on state-changing actions 14

M6: Missing security headers and HTTPS downgrade on static paths 15

I1: Server technology and version disclosure 16

5 Disclaimer 17

A Appendix 18

A.1 Static Appendix Section 18

A.2 Severity Classification 18

List of Figures

Figure 1 - Distribution of identified vulnerabilities 6

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report	Oct 6, 2026

2 Executive Summary

2.1 Overview

Executive Summary

ClawSec conducted a security assessment of **Imagur** (also operating as "FShare" / "Image Sharer"), a public-facing image-sharing web application at imagesharer.xyz, on 6 October 2026. The application runs on a single virtual private server and lets users create accounts, upload images, and share them publicly or keep them private.

The overall security posture is **moderate**. The application correctly requires users to sign in and, for the most part, correctly keeps private images out of other users' reach. However, the assessment confirmed **8 findings**: one high-severity, six medium-severity, and one informational.

The most important issue is a **stored cross-site scripting (XSS) vulnerability** in the image filename field. In practical terms, an attacker can upload an image whose filename contains hidden code; when any visitor (even someone who is not logged in) opens that image, the code runs in their browser. Because the application's login session is not properly protected, this can be escalated to taking over another user's account. This should be fixed first.

The remaining medium-severity issues concern how the application protects user actions and data: it does not defend against cross-site request forgery, its session cookie is missing important security flags, and image upload trusts a hidden field that lets an attacker place images in another user's account. In addition, the underlying database (PostgreSQL) is directly reachable from the internet, important security headers (HSTS and Content-Security-Policy) are missing, and the registration page reveals whether an email address already has an account.

One informational finding notes that the application discloses detailed server and software versions, which can aid a determined attacker.

No testing was destructive, and no sensitive data was accessed or exfiltrated. We recommend addressing the high-severity XSS and the session-cookie hardening immediately, followed by the access-control and CSRF fixes, and finally the remaining hardening items.

Risk Overview

Severity	Count
Critical	0
High	1
Medium	6
Low	0
Info	1

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
H1	7.6	Stored Cross-Site Scripting (XSS) via image filename	9
M1	5.4	Session cookie missing HttpOnly, Secure, and SameSite flags	10
M2	5.3	PostgreSQL service exposed to the internet without TLS	11
M3	5.3	User account enumeration via registration	12
M4	4.3	Broken Access Control / Mass Assignment via user_id on upload	13
M5	4.3	Cross-Site Request Forgery (CSRF) on state-changing actions	14
M6	4.3	Missing security headers and HTTPS downgrade on static paths	15
I1	0.0	Server technology and version disclosure	16

Vulnerability Overview

In the course of this penetration test **1 High**, **6 Medium** and **1 Info** vulnerabilities were identified:

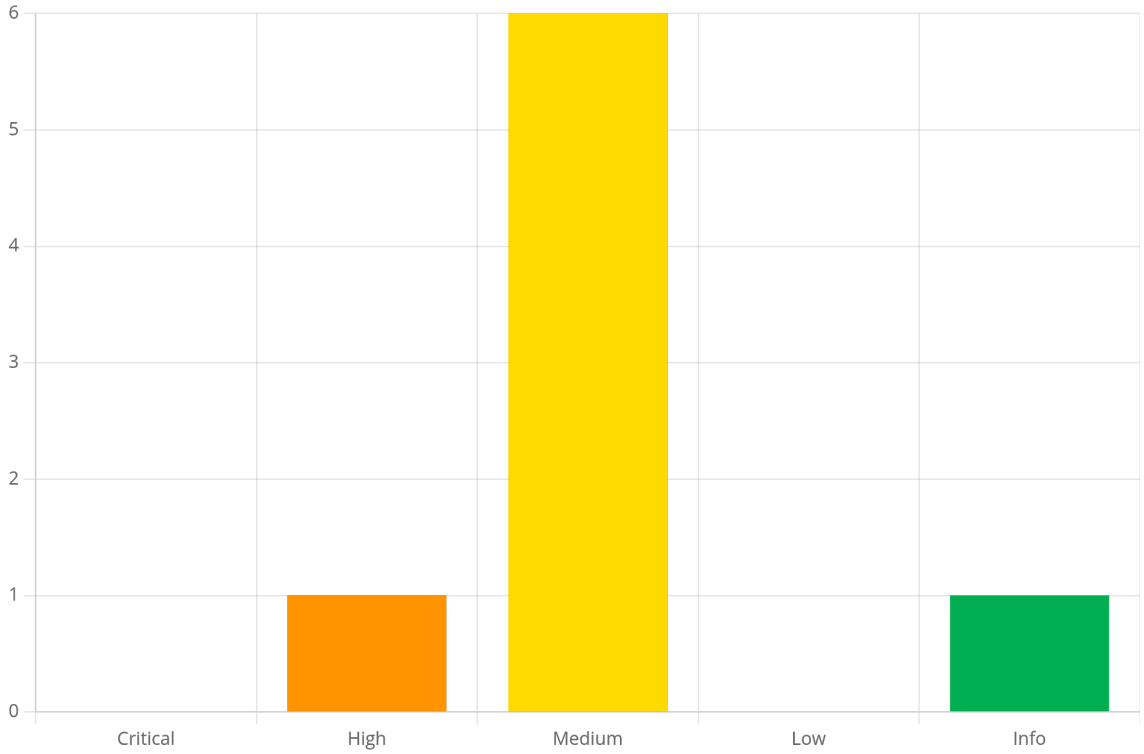


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

Scope

The following systems were assessed on 6 October 2026.

System	Description
imagesharer.xyz	Main public-facing "Imager" image-sharing web application (nginx + PHP 8.2.34)
www.imagesharer.xyz	Alias of the main website (same server)
51.161.131.44	OVH VPS hosting the application - ports 22 (OpenSSH 10.0p2), 80, 443 (nginx), 8080 (Apache 2.4.68), 5432 (PostgreSQL)
51.161.131.44:5432	Exposed PostgreSQL database service

The following assets were **out of scope** and were not tested: resources.imagesharer.xyz (CloudFront/S3), the apex IPv6 address (Optus residential), and third-party infrastructure (AWS CloudFront, OVH, Spaceship).

Methodology

1. Authorization verification (authorization header check on the main website).
2. Passive OSINT, DNS, and certificate-transparency enumeration.
3. Fast TCP port scan followed by version/service fingerprinting.
4. Web application testing: authentication, upload, IDOR/access control, injection, XSS, CSRF, session and security-header review.
5. Targeted PostgreSQL protocol probing.

All testing was non-destructive. No brute force, credential stuffing, or exploitation beyond the minimum proof-of-concept was performed.

Coverage Summary

- Full TCP port scan of the in-scope host.
- Web endpoints: /login, /register, /dashboard, /profile, /images/upload, /images/ (detail, visibility, delete), /profile/avatar.
- Upload validation and filename handling.
- Cross-account access control.
- SQL injection spot-checks on login/register (negative).
- Session cookie and HTTP header review.

Limitations

- SQL injection testing was limited to manual payloads on login/register (no automated scanner such as sqlmap).
- No authenticated rate-limiting or brute-force testing was performed (outside allowed scope).
- The S3 storage bucket and CloudFront distribution were treated as out of scope.
- WHOIS registrant data was privacy-protected.

3.3 User Accounts and Permissions

- No test accounts were provided; assessment was performed as an unauthenticated and self-registered user (black-box).

4 Findings

H1: Stored Cross-Site Scripting (XSS) via image filename	
Score	7.6 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N
Target	https://imagesharer.xyz/ (image detail page /images/<uuid>)
References	https://owasp.org/www-community/attacks/xss/

Overview

The image upload feature stores the user-supplied filename and renders it without HTML escaping in the "Filename" metadata field on the image detail page. An attacker can upload an image whose filename contains HTML/JavaScript, producing stored XSS that executes in the browser of anyone who views the image. Because images can be marked "public" and public images are viewable without authentication, this is effectively an unauthenticated stored XSS.

Details

During testing, a file was uploaded with the filename `xss<img src=x onerror=alert(1)>.png` and valid PNG content. The upload was accepted and the detail page rendered the filename as a live element: `<span>xss<img src=x onerror=alert(1)>.png</span>` (raw, unescaped), while the same value was correctly HTML-escaped in the contexts. The `<img ... onerror=...>` payload therefore executes when the page loads. Confirmed public images return HTTP 200 to unauthenticated requests, so a malicious filename on a public image executes for any visitor. Impact is compounded by the session cookie (PHPSESSID) lacking the HttpOnly flag, allowing the injected script to steal the victim's session and take over their account.

Recommendation

Escape all user-controlled output (htmlspecialchars / context-aware encoding) when rendering the filename in every context, including the metadata "Filename" field. As defense in depth, set the PHPSESSID cookie with HttpOnly, add a Content-Security-Policy, and validate/sanitize filenames on upload (restrict to a safe character set).

M1: Session cookie missing HttpOnly, Secure, and SameSite flags	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	https://imagesharer.xyz/
References	https://owasp.org/www-community/HttpOnly

Overview

The PHPSESSID session cookie is set with only `path=/` and lacks the HttpOnly, Secure, and SameSite attributes, weakening session security.

Details

Every authenticated response sets `Set-Cookie: PHPSESSID=...; path=/` with no HttpOnly, Secure, or SameSite flag. Consequences: (1) the session identifier is readable by JavaScript, so the confirmed stored-XSS vulnerability can be trivially escalated to full session/account takeover; (2) the cookie is transmitted over plaintext HTTP if the user ever connects over HTTP, and is not protected by HSTS; (3) the cookie is sent on cross-site requests, which enables the CSRF finding above.

Recommendation

Set PHPSESSID with `HttpOnly; Secure; SameSite=Lax` (and consider `SameSite=Strict`), and configure the PHP session cookie parameters accordingly (session.cookie_httponly, session.cookie_secure, session.cookie_samesite).

M2: PostgreSQL service exposed to the internet without TLS	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	51.161.131.44:5432
References	https://www.postgresql.org/docs/current/auth-pg-hba-conf.html

Overview

The database server (PostgreSQL) is directly reachable from the public internet on port 5432 and does not offer TLS on the connection.

Details

Port 5432 is open on the public web-server IP 51.161.131.44. A PostgreSQL protocol probe confirmed a PostgreSQL server (postmaster.c) that responds to an SSLRequest with 'N' (no TLS) and requires authentication via SASL SCRAM-SHA-256 (unauthenticated access is not possible). While authentication is enforced, exposing the database directly to the internet unnecessarily expands the attack surface and transmits all traffic (including query results) without transport encryption.

Recommendation

Restrict access to port 5432 to the application host only (e.g., bind to localhost or use a security group/ firewall), and enable TLS on PostgreSQL connections. Use a strong, unique database password and least-privilege database roles.

M3: User account enumeration via registration	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://imagesharer.xyz/register
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/03-Identity_Management_Testing/04-Testing_for_Account_Enumeration_and_Guessable_User_Account

Overview

The registration endpoint reveals whether a given email address is already registered, allowing an attacker to enumerate valid accounts.

Details

Submitting an already-registered email returns the message "An account with that email already exists." while a new email succeeds. This difference allows an unauthenticated attacker to confirm which email addresses have accounts on the service.

Recommendation

Use a generic response for both outcomes (e.g., "If that address is valid, a confirmation email has been sent") and/or rate-limit the registration endpoint.

M4: Broken Access Control / Mass Assignment via user_id on upload	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
Target	https://imagesharer.xyz/images/upload
References	https://cwe.mitre.org/data/definitions/639.html

Overview

The image upload endpoint trusts a client-supplied `user_id` hidden field. An authenticated attacker can substitute another user's UUID and upload images that are attributed to that victim's account (object-creation IDOR / mass assignment).

Details

The upload form contains `<input type="hidden" name="user_id" value="<uuid>">`. By replacing this value with a victim user's UUID, an upload was accepted (HTTP 302 to /images/), and the resulting image appeared in the victim's dashboard while the attacker could not view it. This demonstrates that the server derives image ownership from the client-controlled field instead of the authenticated session. An attacker can inject arbitrary images (including images with malicious filenames, see the XSS finding) into another user's account and gallery.

Recommendation

Ignore the client-supplied `user_id` and derive the owner from the authenticated session on the server. Do not expose a mutable user identifier in the request body.

M5: Cross-Site Request Forgery (CSRF) on state-changing actions	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
Target	https://imagesharer.xyz/ (login, register, logout, image upload, image delete, image visibility, avatar upload)
References	https://owasp.org/www-community/attacks/csrf

Overview

None of the application's forms include a CSRF token, and the session cookie (PHPSESSID) is issued without a SameSite attribute. This allows an attacker to forge cross-site requests that perform actions on behalf of a logged-in victim.

Details

All observed forms (login, register, logout, image upload, image delete, image visibility toggle, and avatar upload) are submitted via POST without any CSRF token or other anti-CSRF control. The PHPSESSID cookie is sent without SameSite, so it will accompany cross-site requests in modern browsers (SameSite is absent, not Lax/Strict). An attacker can therefore force a victim to log out, upload an avatar, delete an image, or change image visibility by enticing them to visit a malicious page.

Recommendation

Add a per-session CSRF token (or SameSite=Strict/Lax + Origin/Referer validation) to every state-changing request. Set SameSite on the session cookie as an additional control.

M6: Missing security headers and HTTPS downgrade on static paths	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N
Target	https://imagesharer.xyz/ (and 51.161.131.44:8080)
References	https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html

Overview

The application does not send HTTP Strict-Transport-Security (HSTS) or a Content-Security-Policy (CSP), and several static paths redirect from HTTPS to plaintext HTTP.

Details

Responses include X-Content-Type-Options, X-Frame-Options, X-XSS-Protection and Referrer-Policy, but not Strict-Transport-Security or Content-Security-Policy. In addition, the `/uploads`, `/css`, and `/js` paths issue a 301 whose Location uses the `http://` scheme (e.g., `http://imagesharer.xyz/uploads/`), downgrading those requests to unencrypted HTTP. The absence of CSP removes a useful mitigation against the confirmed stored-XSS finding.

Recommendation

Add `Strict-Transport-Security: max-age=31536000; includeSubDomains` and a restrictive Content-Security-Policy. Fix the trailing-slash redirects to preserve the `https://` scheme and correct the server's scheme detection behind the reverse proxy.

I1: Server technology and version disclosure	
Score	0.0 (Info)
Vector string	N/A
Target	https://imagesharer.xyz/ (nginx), 51.161.131.44:8080 (Apache)
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework

Overview

The application discloses detailed server and runtime versions that aid targeted attacks.

Details

Responses include `X-Powered-By: PHP/8.2.34`, `Server: nginx`, and on port 8080 `Server: Apache/2.4.68 (Debian)`. SSH also discloses `OpenSSH 10.0p2 Debian`. This version information makes it easier for an attacker to identify applicable exploits.

Recommendation

Remove or obfuscate the X-Powered-By header (`expose_php = Off`), set `server_tokens off` in nginx and `ServerTokens Prod` in Apache, and keep all software patched.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Severity Classification

Severity Classification

Findings in this report are rated using the Common Vulnerability Scoring System (CVSS) v3.1 and grouped into the following categories.

Severity	CVSS v3.1 Base Score	Meaning
Critical	9.0 - 10.0	Immediate, high-impact issues that should be remediated as a priority.
High	7.0 - 8.9	Serious issues that can lead to significant compromise.
Medium	4.0 - 6.9	Notable weaknesses that should be addressed, but are harder to exploit or less severe in impact.
Low	0.1 - 3.9	Minor issues that provide limited value to an attacker.
Info	N/A	Informational observations with no direct security impact.